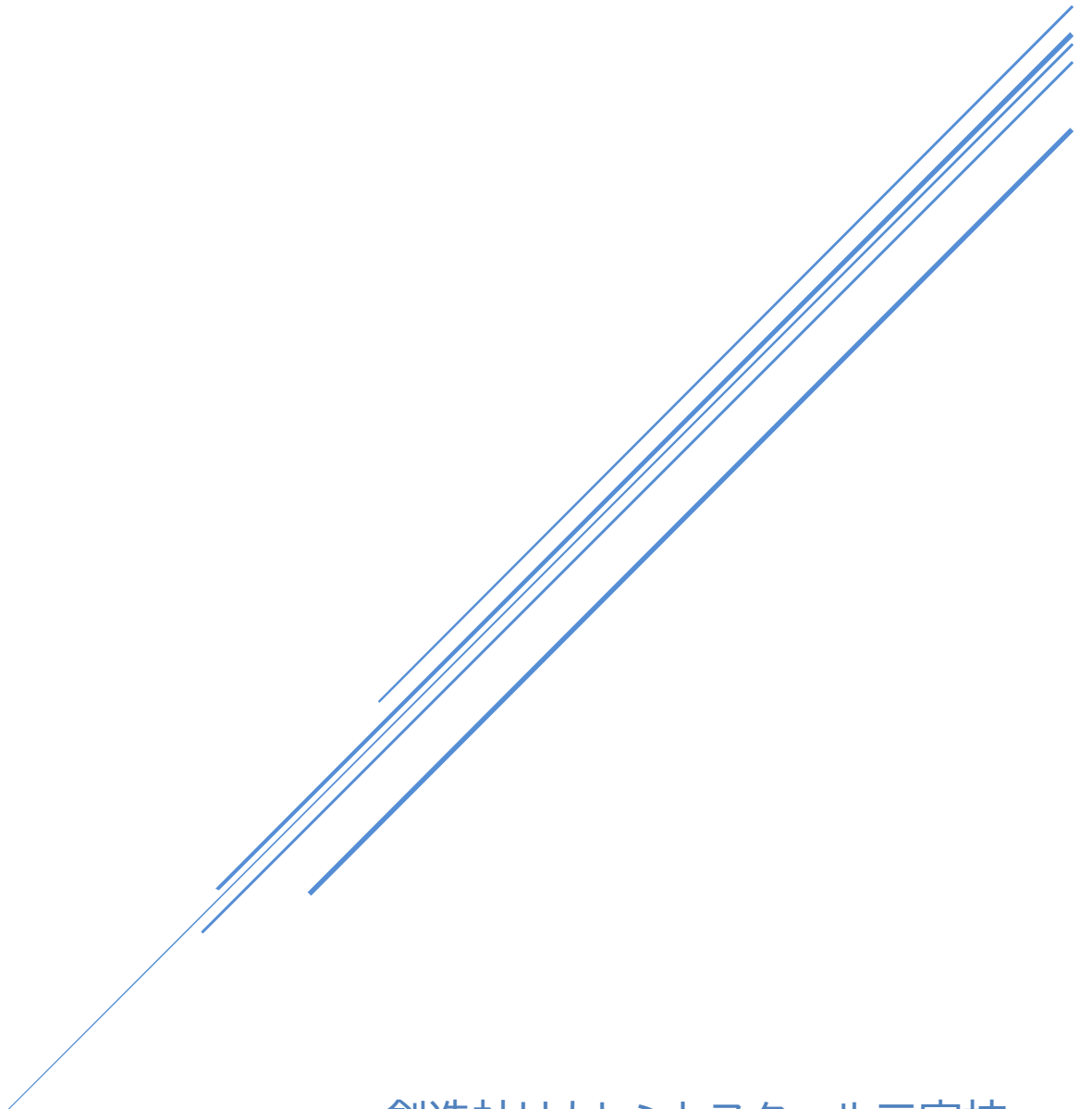


Git の操作マニュアル(生徒向け)

Git/GitHub の利用【2025 年 11 月 25 日】



創造社リカレントスクール三宮校
Java+Python プログラマ養成講座コース

Git 操作マニュアル（生徒向け）

目次

1	はじめに.....	2
2	Git は何ができるか.....	2
3	Git の基本用語.....	2
4	Git のファイル管理.....	2
5	Git の構成.....	3
6	個人ブランチでの運用.....	3
6.1	ブランチの作成.....	3
6.2	最新教材の取得.....	4
6.3	成果物やメモを残すフォルダ.....	4
7	Git の主なコマンド一覧.....	6
8	Git 初心者がやりがちな誤操作一覧.....	6
9	Git の取り消し・やり直し方法.....	6

1 はじめに

目的：授業で Git を使う理由は、コードのバージョン管理と共同開発を円滑にするためです。

運用方針：最初は生徒ごとにブランチを作成し、企画実習・制作等はチーム単位で新しいリポジトリを作成します。

2 Git は何ができるか

- ・プログラムコードを変更前に状態に戻す事ができる。
- ・誰が、いつ、何を変更したかが分かる。
- ・複数の人が同じファイルを編集しても、競合を解決できる。
- ・リモートにプログラムコードを保管できる。
- ・並行して作業ができる。

3 Git の基本用語

リポジトリ	：	ソースコードと履歴を保存する場所
ブランチ	：	並行作業のための分岐
HEAD	：	現在の作業位置
アッド	：	変更をステージングへ登録
ステージング	：	コミット前の一時領域
コミット	：	変更を記録する単位（メッセージ付き）
プッシュ	：	リモートブランチへ登録
プル	：	リモート変更を反映
フェッチ	：	リモート情報を取得
マージ	：	ブランチを統合

4 Git のファイル管理

コミットごとに「その時点のファイルの状態」をスナップショットとして保存するので、
過去に戻れる、変更履歴を追える、安全に試せるというメリットがあります。

5 Git の構成



6 個人ブランチでの運用

6.1 ブランチの作成

Git Bash で以下を実行します。

(ア)「リカレントスクール三宮校」へ移動します。

```
cd "C:\users\ユーザー名\リカレントスクール三宮校\202511"
```

```
$ cd "C:\Users\田 淵 晃\リカレントスクール三宮校(講師)\202511"
```

(イ)ブランチを作成します。

```
git switch -c 出席番号2ケタ-姓-名
```

```
$ git switch -c 99-tabuchi-akira
Switched to a new branch '99-tabuchi-akira'
```

※パスの末尾の () が作業するブランチを指します。

(ウ)リモートに生徒用ブランチを登録します。

```
git push -u origin 出席番号2ケタ-姓-名
```

```
AzureAD+田 淵 晃@TABUCHI-SURFACE-PRO9 MINGW64 ~/リカレントスクール三宮校/202511 (99)
$ git push -u origin 99-tabuchi-akira
total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for '99-tabuchi-akira' on GitHub by visiting:
remote:   https://github.com/Recurrent-S1/202511/pull/new/99-tabuchi-akira
remote:
To https://github.com/Recurrent-S1/202511.git
 * [new branch]      99-tabuchi-akira -> 99-tabuchi-akira
branch '99-tabuchi-akira' set up to track 'origin/99-tabuchi-akira'.
```

(エ)今後は、**生徒用ブランチ**で作業を行うようにしてください。

6.2 最新教材の取得

Origin の main ブランチから最新の教材を取得します。(生徒用ブランチで実行)

(ア)情報を取得します。

```
git fetch origin main
```

```
$ git fetch origin
remote: Enumerating objects: 8, done.
remote: Counting objects: 100% (8/8), done.
remote: Compressing objects: 100% (6/6), done.
remote: Total 7 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (7/7), 6.87 KiB | 390.00 KiB/s, done.
From github-S:Recurrent-S1/202511
 5509be8..6bd8883  main    -> origin/main
```

(イ)ローカルリポジトリと差分があるか確認します。

```
git log HEAD..origin/main --oneline
```

```
$ git log HEAD..origin/main --oneline
6bd8883 (origin/main, origin/HEAD) マニュアル用の DDL
```

差分があれば表示されます。

(ウ)生徒用ブランチに最新の教材を反映させます。

```
git merge origin/main
```

```
$ git merge origin/main
Updating 5509be8..6bd8883
Fast-forward
 .../RPG\343\203\207\343\203\274\343\202\277.txt" | 301 ++++++
 ...02\277\343\203\231\343\203\274\343\202\271.txt" | 158 ++++++
 ...02\277\343\203\231\343\203\274\343\202\271.txt" | 111 ++++++
 3 files changed, 570 insertions(+)
 create mode 100644 "...\343\202\271\343\203\203\343\202\255\343\203\252\343\202\217\343\202\213SQL\345\205\245\351\226\200\347\254\254\357\274\224\347\211\210\343\203\211\343\203\253\dd1\RPG\343\203\207\343\203\274\343\202\277.txt"
 create mode 100644 "...\343\202\271\343\203\203\343\202\255\343\203\252\343\202\217\343\202\213SQL\345\205\245\351\226\200\347\254\254\357\274\224\347\211\210\343\203\211\343\203\253\dd1\345\212\200\350\241\214\345\217\243\345\272\247\343\203\207\343\203\274\343\203\231\343\203\274\343\202\271.txt"
 create mode 100644 "...\343\202\271\343\203\203\343\202\255\343\203\252\343\202\217\343\202\213SQL\345\205\245\351\226\200\347\254\254\357\274\224\347\211\210\343\203\211\343\203\253\dd1\345\212\200\350\241\214\345\217\243\345\272\247\343\203\207\343\203\274\343\203\231\343\203\274\343\202\271.txt"
```

※差分がある場合です。

6.3 成果物やメモを残すフォルダ

個人的に保管しておくフォルダを作成します。(生徒用ブランチで実行)

(ア)WORK フォルダを作成します。

```
mkdir WORK
```

```
$ mkdir WORK
```

(イ)メモ帳を作成します。

```
echo "ここは成果物やメモなどを置くフォルダ~" > WORK/memo.txt
```

```
AzureAD+田 淵 晃 @TABUCHI-SURFACE-PRO9 MINGW64 ~/リカレント  
$ echo "ここは成果物やメモなどを置くフォルダ~" > WORK/h
```

(ウ)ステージングエリアへ追加します。

```
git add .
```

```
AzureAD+田 淵 晃 @TABUCHI-SURFACE-PRO9 MINGW64 ~/リカレントスクール三宮校  
$ git add .  
warning: in the working copy of 'WORK/memo.txt', LF will be replaced by  
CRLF when it is committed
```

(エ)ローカルリポジトリへ登録します。

```
git commit -m "作業用フォルダの作成"
```

```
AzureAD+田 淵 晃 @TABUCHI-SURFACE-PRO9 MINGW64 ~/リカレント  
$ git commit -m "作業用フォルダの作成"
```

(オ)リモートリポジトリの生徒ブランチへ登録します。

```
git push origin 出席番号2ケタ-姓-名
```

```
AzureAD+田 淵 晃 @TABUCHI-SURFACE-PRO9 MINGW64 ~/リカレントスクール三宮校  
$ git push origin 99-tabuchi-akira  
Enumerating objects: 5, done.  
Counting objects: 100% (5/5), done.  
Delta compression using up to 12 threads  
Compressing objects: 100% (3/3), done.  
Writing objects: 100% (4/4), 446 bytes | 446.00 KiB/s, done.  
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)  
To https://github.com/Recurrent-S1/202511.git  
45415a1..eca42e1 99-tabuchi-akira -> 99-tabuchi-akira
```

(カ)今後、何かファイルを保管したければ、上記の手順で登録ができます。

7 Git の主なコマンド一覧

操作カテゴリ	コマンド例	説明
初期設定	git config --global user.name "名前"	ユーザー名を設定
	git config --global user.email "メール"	メールアドレスを設定
リポジトリ作成	git init	新しいローカルリポジトリを作成
	git clone URL	リモートリポジトリを複製
状態確認	git status	現在の状態を確認(変更・ステージング状況)
	git log	コミット履歴を表示
ステージング	git add ファイル名	ファイルをステージング
	git add .	すべての変更をステージング
コミット	git commit -m "メッセージ"	ステージングした変更をコミット
ブランチ操作	git branch	ブランチ一覧を表示
	git checkout -b ブランチ名	新しいブランチを作成して切り替え
	git switch ブランチ名	ブランチを切り替え(推奨)
リモート操作	git remote add origin URL	リモートリポジトリを追加
	git push origin ブランチ名	リモートに変更を送信
	git pull origin ブランチ名	リモートの変更を取得してマージ
マージ	git merge ブランチ名	指定ブランチを現在のブランチに統合
一時退避	git stash	作業中の変更を一時的に保存

8 Git 初心者がやりがちな誤操作一覧

操作	誤操作内容	注意点
Add	git add . で不要なファイルまで追加	git status で確認、.gitignore を設定
Commit	意味のないメッセージ (例: update)	「何を」「なぜ」変更したかを書く
	大量の変更を 1 コミットにまとめる	小さな単位でコミット
Push	git push -f で履歴を壊す	強制 push は原則禁止
	pull せずに push して競合発生	push 前に git pull で最新を取り込む
Pull	未コミット変更を上書きしてしまう	pull 前に git status で確認
	競合発生時に reset --hard で履歴を消す	慌てず競合解消、merge --abort で戻す
Marge	main ブランチに直接 merge	ブランチ戦略に沿って merge
	コンフリクト解消時に誤ってコード削除	解消後に動作確認
その他	git reset --hard で作業内容を完全消去	慎重に使用、必要なら stash で退避
	rebase で履歴を壊す	チーム開発では慎重に

9 Git の取り消し・やり直し方法

操作対象	コマンド例	説明
ステージング前の変更を取り消す	git restore ファイル名	作業ディレクトリの変更を元に戻す (未ステージングの変更を破棄)
ステージングを解除	git restore --staged ファイル名	git add したファイルをステージから外す
最後のコミットを修正	git commit --amend	直前のコミットメッセージや内容を修正

コミットを取り消す（履歴残す）	<code>git revert コミット ID</code>	指定したコミットを「打ち消す」新しいコミットを作成
コミットを取り消す（履歴も消す）	<code>git reset --soft コミット ID</code>	指定コミットまで戻し、変更はステージング状態で残す
コミットとステージングを取り消す	<code>git reset --mixed コミット ID</code>	指定コミットまで戻し、変更は作業ディレクトリに残す
完全に取り消す（危険）	<code>git reset --hard コミット ID</code>	指定コミットまで戻し、変更を完全に破棄
未コミットの変更を一時退避	<code>git stash</code>	作業中の変更を一時的に保存し、クリーンな状態に戻す
リモートの push を取り消す	<code>git push origin ブランチ名 --force</code>	強制的にリモートを上書き（履歴改変、チーム開発では注意）

おわり